UNIVERSIDADE DE SÃO PAULO · ESCOLA POLITÉCNICA

Polling, Interrupções, Threads e Work Queue no Zephyr RTOS

Relatório do hands-on: medição de uso de CPU, memória e tempo de execução em ESP32

Aluno: Leonardo Moreno · NUSP 13654481
Data: 19/10/2026
Roteiro: Apostila "Polling, Interrupções, Threads e Workqueue no Zephyr RTOS", v1.0 (set/2026)
Código-base: github.com/caninos-loucos/zephyr-esp32-lab

1. Objetivo

Resolver o mesmo problema (detectar o acionamento de um botão e tratá-lo) com três arquiteturas do Zephyr RTOS no ESP32: **polling**, **interrupção + thread com semáforo** e **interrupção + work queue**. Em seguida, comparar as três com números de tempo (latência de detecção, hand-off, duração, jitter), CPU (ocupação total e por thread) e memória (Flash, RAM, pilhas e objetos do kernel). Também foram estudados o efeito do período de polling, da carga e da taxa de eventos, o anti-padrão de processar dentro da ISR e um sistema com processamento de 100 ms por evento, para justificar uma escolha de arquitetura.

2. Ambiente

Tabela 2.1 - Ambiente de compilação e execução

Item	Valor
Placa alvo (build)	`esp32_devkitc/esp32/procpu` (ESP32-DevKitC, núcleo PRO)
Zephyr	4.5.0-rc2, commit `03b1ebca3343` (manifesto do repositório do curso, revisão `main`)
Zephyr SDK / toolchain	1.0.1 / `xtensa-espressif_esp32_zephyr-elf-gcc` 14.3.0
HAL Espressif	`hal_espressif` commit `c1e43fdd`; west 1.5.0
CPU / contador	Xtensa LX6 a 240 MHz; `hw cycles/s = 240000000` (resolução de 4,17 ns); tick = 1000 Hz
Temporizador do sistema	`CONFIG_XTENSA_TIMER` (CCOUNT/CCOMPARE)
Execução	ESP32 em emulação (QEMU Espressif) com o firmware compilado para `esp32_devkitc`; detalhes no Apêndice B

Tabela 2.2 - Ligações do banco de medição

Função	Pino	Ligação	Configuração (app.overlay)	Uso
LED	GPIO2	GPIO2 → LED → GND	saída, ativo em nível alto	alterna a cada evento tratado
Botão	GPIO4	GPIO4 → botão → GND	entrada, ativo em nível baixo, *pull-up* interno	evento externo (Exps. 1 a 3)
Estímulo	GPIO18	GPIO18 → jumper → GPIO4	saída, ativo em nível baixo	gerador de eventos (Exps. 4 a 8)

3. Metodologia de medição

3.1 Instrumentação (igual à da apostila)

O código do repositório do curso foi usado sem alterações nos experimentos 1 a 7. Os pontos `t_stim` (na `main`, antes de acionar o pino de estímulo), `t_isr` (primeira linha da ISR ou instante de detecção no polling), `t_ini` (início da thread ou do work handler) e `t_fim` (fim de `app_process_event()`) são lidos com `k_cycle_get_32()` e convertidos com `k_cyc_to_ns_floor64()`. A CPU ocupada vem de dois instantâneos de `k_thread_runtime_stats_all_get()` (100 % − idle); a CPU por thread e as pilhas vêm do Thread Analyzer; a memória estática vem do resumo do `west build` e de `ram_report`/`rom_report`. Cada benchmark usa 5 eventos de aquecimento e 200 eventos medidos, sem `printk` na janela.

3.2 Banco de estímulo

Nos Exps. 4 a 8 a `main` gera pulsos no GPIO18, que chegam ao GPIO4 pelo jumper (padrão: 200 pulsos de 20 ms a cada 97 ms, após 5 de aquecimento). O instante `t_stim` é lido imediatamente antes de cada pulso. Nos Exps. 1 a 3 o GPIO4 recebeu uma sequência roteirizada de 25 toques de 80 a 220 ms, separados por 0,4 a 1,5 s; a cada cinco, um toque rápido de 4 ms, mais curto que o período de polling. As transições dos pinos GPIO2, GPIO4 e GPIO18 também foram registradas com carimbo de tempo, como num analisador lógico (`logs/*_gpio_trace.csv`). Cada configuração foi executada 3 vezes nos Exps. 4 e 5 e uma vez nos demais.

3.3 Validação com medida externa

O registro dos pinos em cada execução do Exp. 4 dá uma medida externa, independente do software: o intervalo entre a borda do estímulo (GPIO18) e a alternância do LED (GPIO2, primeira ação de `app_process_event()`), equivalente à Extensão E7.

Tabela 3.1 - Medida interna × medida externa (Exp. 4, execução 1)

Modo	lat_process (software), µs	Estímulo → LED (pinos), µs	Diferença, µs
Polling	8.999,92	9.000,24	0,32
ISR + thread	4,42	4,63	0,21
ISR + work queue	5,03	5,24	0,21

A diferença positiva e constante é o viés de `gpio_pin_set_dt()` entre `t_stim` e a borda, somado ao tempo até a alternância do LED; ela não altera a comparação entre os modos.

3.4 Achado: o arredondamento de `k_msleep()` trava a fase do polling

Na primeira execução do polling a latência de detecção saiu praticamente constante (8,98 a 9,00 ms), e não uniforme entre 0 e 10 ms como prevê a apostila. O registro dos pinos mostrou que o estímulo se repete a cada **99 ms**, não 97. A causa está em `kernel/timeout.c`: um timeout relativo é arredondado **+1 tick** para garantir "pelo menos N ticks". Assim `k_msleep(20)` + `k_msleep(77)` duram 21 + 78 = 99 ticks, e o laço de polling com `k_msleep(10)` roda a cada 11 ticks. Como 99 = 9 × 11, o estímulo cai sempre na mesma fase do polling. É um comportamento do kernel, independente da placa. Para os experimentos que dependem da varredura de fase (Exp. 5 e a Tabela 10.1b) o período foi ajustado para `APP_EVENT_PERIOD_MS=95`, que resulta em 97 ticks reais (primo).

3.5 Fontes de erro consideradas

- Viés do estímulo (`gpio_pin_set_dt()` entre `t_stim` e a borda), constante nos modos (Tabela 3.1).
- Resolução do contador: 4,17 ns; estouro de 32 bits em 17,9 s (só diferenças curtas são medidas).
- Cache e flash nas primeiras execuções de cada trecho de código (mitigados pelo aquecimento).
- Tick de 1 ms e o arredondamento +1 tick (seção 3.4).
- Tempo de ISR contabilizado na thread interrompida (CPU "ocupada" subestimada em ISRs longas, Exp. 7).
- Coalescência de eventos (semáforo com limite 1 e work já pendente), que transforma eventos em perdas.

4. Implementação do polling

Uma thread `poll` (prioridade 1, pilha de 1 KiB) lê o pino a cada `APP_POLL_PERIOD_MS` e trata a borda de subida do estado lógico; `last` evita contar o mesmo toque várias vezes. O `k_msleep()` suspende a thread, liberando a CPU para a idle.

```
static void poll_thread(void *p1, void *p2, void *p3) {

    int last = 0;

    while (1) {
        int state = gpio_pin_get_dt(&button); /* 1 = pressionado */

        if (state > 0 && last == 0) { /* borda de subida */
            uint32_t now = k_cycle_get_32();
            atomic_inc(&ev_detected);

            if (measuring) {
                lat_add(&lat_detect, now - t_stim);
                lat_add(&lat_process, now - t_stim);
            }
            app_process_event();
        }
        last = state;

        k_msleep(CONFIG_APP_POLL_PERIOD_MS);
```

5. Implementação da interrupção + thread

A ISR só registra `t_isr`, conta o evento e libera um semáforo `K_SEM_DEFINE(button_sem, 0, 1)`. A thread `worker` (prioridade 1, 1 KiB) fica bloqueada em `k_sem_take(K_FOREVER)`. A ordem de inicialização é callback → `gpio_add_callback()` → `gpio_pin_interrupt_configure_dt(GPIO_INT_EDGE_TO_ACTIVE)`.

```
static void button_isr(const struct device *port, struct gpio_callback *cb,
                                            gpio_port_pins_t pins) {
    uint32_t now = k_cycle_get_32(); /* <- ponto t_isr */
    t_isr = now;

    atomic_inc(&ev_detected);

    if (measuring) {
        lat_add(&lat_detect, now - t_stim);
    }
    k_sem_give(&button_sem);
}

static void worker(void *p1, void *p2, void *p3) {
    while (1) {
        k_sem_take(&button_sem, K_FOREVER); /* bloqueada sem evento */

        uint32_t now = k_cycle_get_32(); /* <- ponto t_ini */
        if (measuring) {
            lat_add(&lat_handoff, now - t_isr);
            lat_add(&lat_process, now - t_stim);
        }
        app_process_event();
```

6. Implementação da work queue

A ISR submete um `k_work` à *system workqueue* (`sysworkq`, mesma prioridade 1 da worker, por `CONFIG_SYSTEM_WORKQUEUE_PRIORITY=1`). Não há thread própria: o handler roda na pilha da sysworkq.

```
static void button_work_handler(struct k_work *work) {
    uint32_t now = k_cycle_get_32(); /* <- ponto t_ini */
    if (measuring) {
        lat_add(&lat_handoff, now - t_isr);
        lat_add(&lat_process, now - t_stim);
    }
    app_process_event();
}

K_WORK_DEFINE(button_work, button_work_handler);
/* ISR: curta. Apenas registra o tempo e submete o work item. */
static void button_isr(const struct device *port, struct gpio_callback *cb,
                                                  gpio_port_pins_t pins) {
    uint32_t now = k_cycle_get_32(); /* <- ponto t_isr */
    t_isr = now;
    atomic_inc(&ev_detected);
    if (measuring) {
        lat_add(&lat_detect, now - t_stim);
    }

    k_work_submit(&button_work); /* retorna 0 se ja estava pendente */
```

7. Resultados

7.1 Experimento 0: validação do ambiente

O `blink_test` compilou sem erros (Flash 143.728 B), deu boot ("`*** Booting Zephyr OS build 03b1ebca3343 ***`") e o registro do GPIO2 mostra o LED alternando a cada 1,0001 s (`logs/exp0_blink_run1.log`). Os 0,1 ms além de 1 s são o tick extra do `k_msleep(1000)` mais o tempo do laço. Itens: compilou ✔; imagem gravada na flash (aplicação em 0x1000) ✔; LED piscando ✔; terminal serial com a mensagem de boot ✔.

7.2 Experimentos 1 a 3: funcionamento em modo manual

Compilação com `-DCONFIG_APP_STIMULUS=n -DCONFIG_APP_VERBOSE=y`; o botão recebeu a sequência de 25 toques da seção 3.2, 5 deles de 4 ms. Cada toque detectado gerou uma linha "evento processado #N" e alternou o LED.

Tabela 7.1 - Observações dos Experimentos 1, 2 e 3 (tabelas 7.1, 8.1 e 9.1 da apostila)

Modo	Toques	Toques de 4 ms	Eventos processados	Não detectados	Pilha da thread do evento (B)
Polling	25	5	22	3	576 / 1024
ISR + thread	25	5	25	0	560 / 1024
ISR + work queue	25	5	25	0	608 / 1024

- **Polling (Exp. 1):** o LED mudou a cada toque detectado e o contador aumentou. Os toques de 4 ms, mais curtos que o período efetivo de 11 ms, foram em parte perdidos. Não houve eventos múltiplos por toque, porque os toques da sequência são limpos, sem *bounce* (num botão mecânico, um clique poderia gerar vários eventos; ver E1). No Thread Analyzer a `poll` aparece com 0 % (arredondado) e a idle com 99 %.
- **Interrupção + thread (Exp. 2):** todos os toques, inclusive os de 4 ms, foram detectados na ISR e processados na `worker`, que fica bloqueada (0 % de CPU) sem eventos. A ISR não faz processamento pesado.
- **Work queue (Exp. 3):** mesmo resultado funcional; o tratamento aparece na thread `sysworkq`. Não existe thread `worker`; o tratamento usa a pilha da sysworkq, que neste Zephyr só é incluída no binário porque o modo C a usa (seção 7.3).

7.3 Experimento 4: benchmark comparativo

Padrões do Kconfig (200 eventos, período 97 ms, pulso 20 ms, polling 10 ms, sem carga), 3 execuções por modo. As três execuções produziram os mesmos valores (Apêndice B), então média e máximo coincidem com os de cada execução.

Tabela 10.1 - Tempo (µs), média das 3 execuções

Métrica (µs)	Polling (P = 10 ms)	ISR + thread	ISR + work queue
lat_detect: média	8.999,92	1,10	1,10
lat_detect: máximo	9.000,01	1,10	1,10
lat_handoff: média	n/a	3,32	3,93
lat_handoff: máximo	n/a	3,33	3,94
lat_process: média	8.999,92	4,42	5,03
lat_process: máximo	9.000,01	4,43	5,04
lat_proc_dur: média	0,18	0,18	0,18
Jitter de lat_process (máx – mín)	16,15	0,01	0,01

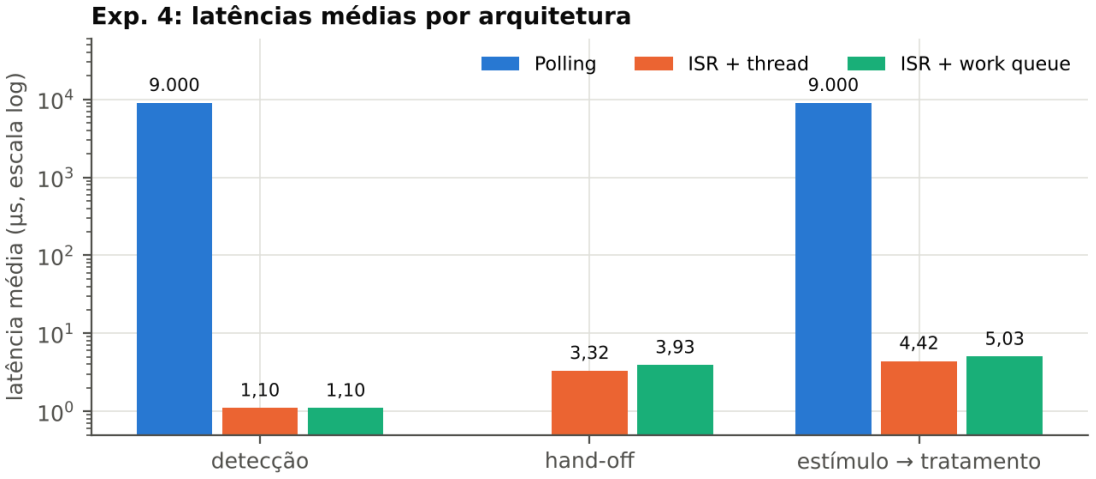


Figura 1 - Latências médias do Exp. 4 (escala logarítmica). O polling detecta o evento cerca de quatro ordens de grandeza depois das arquiteturas por interrupção.

Tabela 10.1b - Polling com a fase varrida

Métrica (µs)	Polling, período 95 ms (97 ticks)
lat_detect: média	6.029,41
lat_detect: mín / máx	1.000,00 / 10.994,31
lat_process: média	6.029,41

Período do estímulo de 95 ms, que resulta em 97 ticks reais (seção 3.4); 3 execuções.

Tabela 10.2 - Eventos

Métrica	Polling	ISR + thread	ISR + work queue
Gerados	200	200	200
Detectados	200	200	200
Processados	200	200	200
Perdidos	0	0	0

Tabela 10.3 - CPU

Métrica	Polling	ISR + thread	ISR + work queue
CPU ocupada (kernel)	0,02 %	0,01 %	0,01 %
CPU estimada (processamento)	0,00 %	0,00 %	0,00 %
idle (Thread Analyzer)	99 %	99 %	99 %
Thread do evento (poll / worker / sysworkq)	0 %	0 %	0 %
main (gerador de estímulo)	0 %	0 %	0 %
Ciclos totais da thread do evento (desde o boot)	1.037.952	151.616	165.537

Tabela 10.4 - Memória

Métrica	Polling	ISR + thread	ISR + work queue
Flash usada (bytes), resumo do build	146.416	146.560	146.496
RAM dram0_0_seg (bytes): dados + bss	12.488	12.520	12.560
RAM .dram0.noinit (bytes): pilhas de todas as threads	8.192	8.192	8.192
RAM estática total (bytes)	**20.680**	**20.712**	**20.752**
IRAM iram0_0_seg (bytes): código em RAM	35.072	35.584	36.352
Pilha da thread do evento: usada / total (B)	256 / 1024	288 / 1024	n/a (usa a sysworkq)
Pilha da sysworkq: usada / total (B)	n/a	n/a	304 / 1024
Pilha da ISR (ISR0): usada / total (B)	352 / 2048	368 / 2048	480 / 2048
Pilha da main: usada / total (B)	1024 / 4096	1024 / 4096	1024 / 4096
sizeof(k_thread) / k_sem / k_work / k_work_q (B)	184 / 16 / 16 / 216	184 / 16 / 16 / 216	184 / 16 / 16 / 216

As pilhas ficam na seção `.dram0.noinit`, que o resumo do build contabiliza na região `dram1_0_seg` arredondada ("8 KB"); o valor exato vem de `objdump -h` (`logs/*_secoes.txt`).

Leitura dos resultados. As hipóteses da seção 10.3 se confirmam, com duas ressalvas (fase do polling e memória). A detecção por interrupção levou 1,10 µs, contra 9,00 ms no polling: cerca de 8.152 vezes mais. Com o padrão de 97 ms, a média do polling não ficou em P/2 por causa da trava de fase (seção 3.4). Com a fase varrida (período 95 ms), a média foi 6,03 ms e o máximo 10,99 ms, coerentes com uma latência distribuída entre 1 e 11 ticks (média ≈ P/2 + 1 tick). O hand-off pela work queue (3,93 µs) foi 0,61 µs mais lento que pelo semáforo (3,32 µs), porque o `k_work_submit()` e o laço da work queue executam mais código que o `k_sem_give()`/`k_sem_take()`. A diferença é pequena, como previsto. O jitter das arquiteturas por interrupção ficou em centésimos de µs, enquanto o do polling é limitado pelo tick.

Em CPU, todas ficaram abaixo de 0,1 %, porque o trabalho útil é quase nulo. A diferença aparece nos ciclos acumulados da thread do evento: a `poll` consumiu 1.037.952 ciclos, 6,8 vezes os 151.616 da `worker`, porque acorda a cada 11 ms mesmo sem evento. A CPU "ocupada" do polling (0,02 %) é o dobro da dos modos por interrupção (0,01 %), mas ambas são uma fração de %, como alerta a apostila.

Memória: a hipótese da apostila não se confirmou nesta versão do Zephyr. Esperava-se que o modo C custasse só o `k_work` (16 B), reaproveitando a pilha da sysworkq "que já existe". No Zephyr 4.5, porém, a system workqueue usa linkagem *pay-per-use*: ela só é incluída no binário quando algo a referencia (comentário em `kernel/system_work_q.c`). Nos builds de polling e de thread não há `sysworkq` (nem no ELF, nem no Thread Analyzer). No modo C ela entra por causa do próprio `k_work_submit()`, trazendo a pilha de 1024 B e o `k_work_q` (216 B). O modo B traz a pilha da worker

(1024 B), o `k_thread` (184 B) e o `k_sem` (16 B). Resultado medido: a RAM estática total foi de 20.680, 20.712 e 20.752 B (polling, thread, work queue); C usou 40 B a **mais** que B. A vantagem de memória da work queue só aparece quando outro driver ou subsistema já usa a sysworkq: aí o custo marginal do modo C cai para o `k_work`, enquanto o de B continua sendo uma pilha inteira. Em Flash as três diferem por poucas centenas de bytes (146.416, 146.560 e 146.496 B): o código de callback e de semáforo/work queue é pequeno perto do kernel e do HAL.

7.4 Experimento 5: efeito do período de polling

Modo polling, 3 execuções por linha, período do estímulo 95 ms (97 ticks reais).

Tabela 11.1 - Varredura do período de polling

P (ms)	Pulso (ms)	Lat. média (µs)	Lat. máx (µs)	CPU ocupada	Detectados / gerados	Perdidos (%)
1	20	1.497,07	1.994,31	0,12 %	200 / 200	0,0
5	20	3.478,98	5.994,31	0,04 %	200 / 200	0,0
10	20	6.029,41	10.994,31	0,02 %	200 / 200	0,0
50	20	10.875,06	21.001,51	0,00 %	80 / 200	60,0
100	20	4.442,65	19.983,85	0,00 %	9 / 200	95,5
1	5	1.497,08	1.994,31	0,12 %	200 / 200	0,0
5	5	3.479,23	5.995,82	0,04 %	200 / 200	0,0
10	5	3.500,25	6.001,52	0,02 %	108 / 200	46,0
50	5	3.391,50	6.001,52	0,00 %	23 / 200	88,5
100	5	2.500,00	4.000,00	0,00 %	8 / 200	96,0

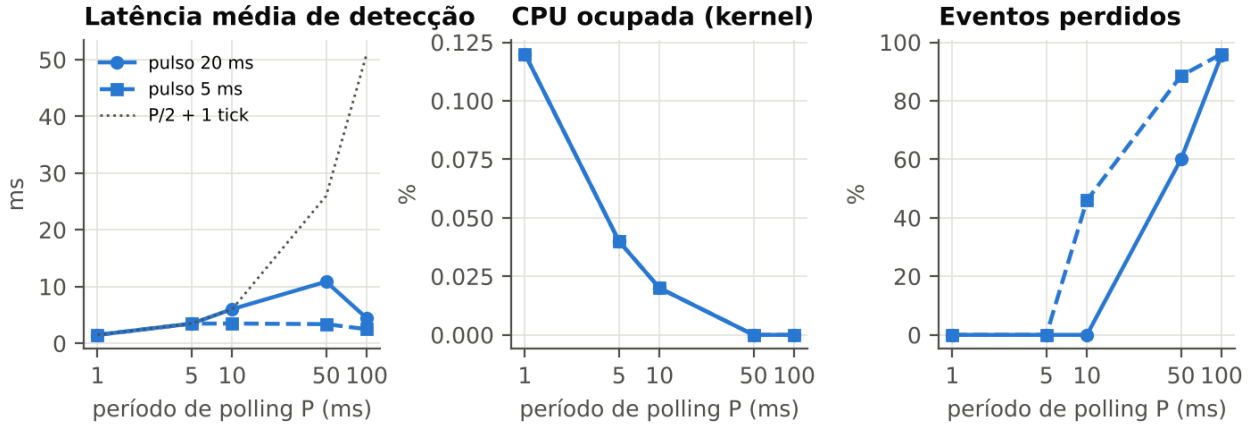


Figura 2 - Latência média de detecção, CPU ocupada e eventos perdidos em função do período de polling (eixo x logarítmico). Linha cheia: pulso de 20 ms; tracejada: pulso de 5 ms; pontilhada: modelo P/2 + 1 tick. A queda da latência em P ≥ 50 se deve às perdas: só os pulsos detectados cedo entram na média.

1. Latência × P e CPU × P. Enquanto o pulso dura mais que o período efetivo, a latência média cresce **linearmente** com P e segue P/2 + 1 tick: 1,50 ms em P = 1, 3,48 ms em P = 5 e 6,03 ms em P = 10, sempre entre 1 tick e P + 1 ticks. O mínimo é de 1 ms, e não 0, porque a `poll` e a `main` acordam no mesmo tick e a `poll`, de maior prioridade, lê o pino antes de a `main` gerar o pulso. Em P = 50 e P = 100 a média fica muito abaixo do modelo (10,88 e 4,44 ms, contra 26 e 51 ms) porque só entram na estatística os pulsos detectados, justamente os que caíram logo antes de uma consulta. A CPU ocupada cai de forma **inversa**, proporcional a 1/(P + 1 tick): o produto CPU × ticks do laço é quase constante (0,12 % × 2 ≈ 0,04 % × 6 ≈ 0,02 % × 11), e em P ≥ 50 ela fica abaixo de 0,01 %. O custo absoluto é pequeno, porque cada consulta dura poucos µs, mas escala com o número de acordadas por segundo.

2. Pulsos de 5 ms sem perda. P = 1 e P = 5 não perderam nenhum pulso; P = 10 perdeu 46,0 %, P = 50 perdeu 88,5 % e P = 100 perdeu 96,0 %. Em P = 5 o laço dura 6 ticks e o pulso de `k_msleep(5)` também dura 6 ticks, então sempre há uma consulta dentro do pulso, mas com margem zero. Eu escolheria **P = 4 ms** (laço de 5 ticks, um tick de margem). O custo fica entre os medidos para P = 5 (0,04 %) e P = 1 (0,12 %).

3. Regra. O período efetivo de polling (P + 1 tick, no Zephyr) deve ser **no máximo igual à largura do menor evento**, e com margem para jitter, **no máximo metade**. Até P = 50 as perdas seguem a previsão 1 − (largura do pulso)/(período do laço): 46,0 % medido contra 45,5 % previsto (pulso de 5 ms, P = 10), e 60,0 % contra 58,8 % (pulso de 20 ms, P = 50). Em P = 100 a perda foi muito maior que a prevista (95,5 % contra 79 %) por **aliasing**. O laço de 101 ticks é quase igual ao período do estímulo (97 ticks), então consultas seguidas caem em pulsos seguidos quase na mesma fase. Se uma consulta encontra o pino pressionado, a seguinte também encontra, `last` continua 1 e a borda não é contada: vários eventos se fundem em um "toque longo". Portanto o período de polling também precisa ser bem menor que o intervalo entre eventos (pelo menos a metade, no espírito de Nyquist).

4. Melhor caso do polling × interrupção. Mesmo em P = 1 a latência média do polling (1,50 ms) é cerca de 1.356 vezes a detecção por interrupção (1,10 µs), e a CPU ocupada (0,12 %) é cerca de 12 vezes a do modo B (0,01 %). Em memória as três empatam neste firmware: cada uma tem uma pilha de 1 KiB para o tratamento (poll, worker ou sysworkq; ver Tabela 10.4).

7.5 Experimento 6: carga de processamento e taxa de eventos

Tabela 12.1 - Carga por evento (período do estímulo = 97 ms)

Carga (µs)	Modo	CPU ocupada	CPU estimada	Lat. process média (µs)	Perdidos	Janela (ms)
0	ISR + thread	0,01 %	0,00 %	4,42	0	19.901
0	ISR + work queue	0,01 %	0,00 %	5,03	0	19.901
1.000	ISR + thread	1,00 %	0,99 %	4,42	0	20.102
1.000	ISR + work queue	1,00 %	0,99 %	5,03	0	20.102
10.000	ISR + thread	9,13 %	9,12 %	4,42	0	21.911
10.000	ISR + work queue	9,13 %	9,12 %	5,03	0	21.911
50.000	ISR + thread	33,39 %	33,38 %	4,42	0	29.951
50.000	ISR + work queue	33,39 %	33,38 %	5,03	0	29.951

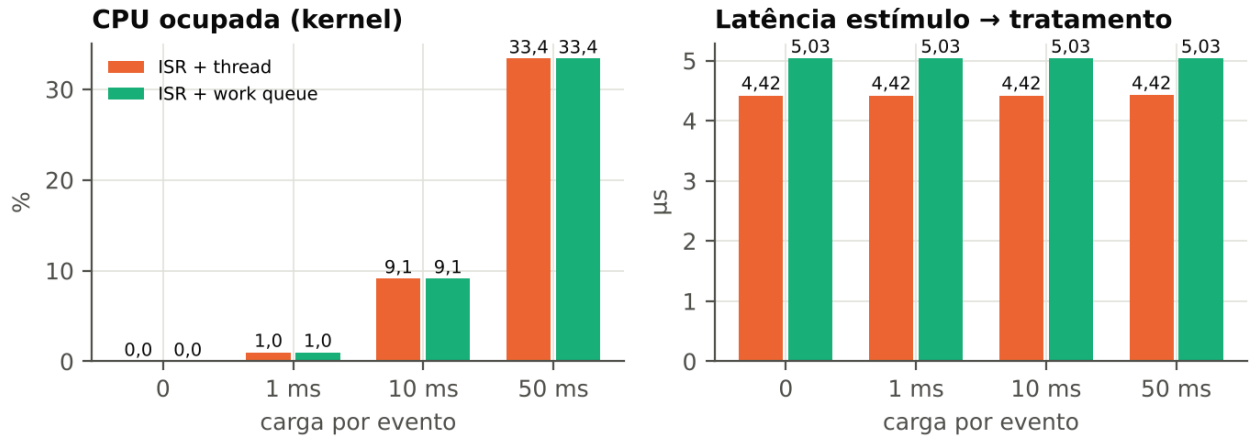


Figura 3 - CPU ocupada e latência até o início do tratamento em função da carga por evento. As duas arquiteturas são indistinguíveis em CPU; a latência não depende da carga.

A CPU ocupada acompanha a CPU estimada a partir das durações (a diferença é o custo do próprio mecanismo) e é igual nos dois modos: o trabalho é o mesmo, muda só quem o executa. A latência até o início do tratamento não depende da carga, porque com período de 97 ms o tratamento anterior já terminou quando chega o próximo evento. A CPU com 10 ms de carga foi 9,13 % e não 10,1 %: a worker (prioridade 1) preempta a `main` logo após o estímulo, antes de ela chamar `k_msleep()`, e assim o período real do estímulo cresce no valor da carga (janela de 21.911 ms em vez de 19.901 ms). Com 100 ms de tratamento (desafio original), uma thread dedicada é preferível: bloquear a sysworkq por 100 ms atrasaria todos os outros work items do sistema (ver Exp. 8).

Tabela 12.2 - Saturação: tratamento de 30 ms, eventos a cada 23 ms (25 ticks reais), main com prioridade 0

Modo	Gerados	Detectados	Processados	Perdidos (%)	CPU ocupada
Polling (P = 10 ms)	200	75	75	62,5	43,87 %
ISR + thread	200	200	167	16,5	97,64 %
ISR + work queue	200	200	167	16,5	97,64 %

- **Detectados > processados nos modos por interrupção.** A ISR vê todas as 200 bordas, mas o encaminhamento coalesce: o semáforo tem limite 1 e um `k_work_submit()` de item já pendente não o enfileira de novo. Foram processados 167 (thread) e 167 (work queue) eventos.
- **Onde ocorre a perda.** Na interrupção, no **encaminhamento** (coalescência), não na detecção. No polling a perda é na **detecção**: enquanto a `poll` processa 30 ms ela não consulta o pino, e só 75 de 200 pulsos foram vistos.
- **Como evitar a perda sem mudar prioridade.** Usar uma fila com um item por evento (`k_msgq`, Extensão E3) ou um semáforo com limite maior que conte os eventos. Isso só ajuda enquanto a fila não enche: com 30 ms de trabalho a cada 25 ms a utilização passa de 100 % e nenhuma arquitetura evita a perda sem reduzir o trabalho.

7.6 Experimento 7: anti-padrão, processamento dentro da ISR

Tabela 13.1 - Impacto no escalonamento

Modo (carga 20 ms)	Maior intervalo do heartbeat (ms)	CPU ocupada (kernel)	CPU estimada	Perdidos
ISR only (anti-padrão)	21,00	0,12 %	16,72 %	0
ISR + thread	2,00	16,82 %	16,72 %	0
ISR + work queue	2,00	16,82 %	16,72 %	0

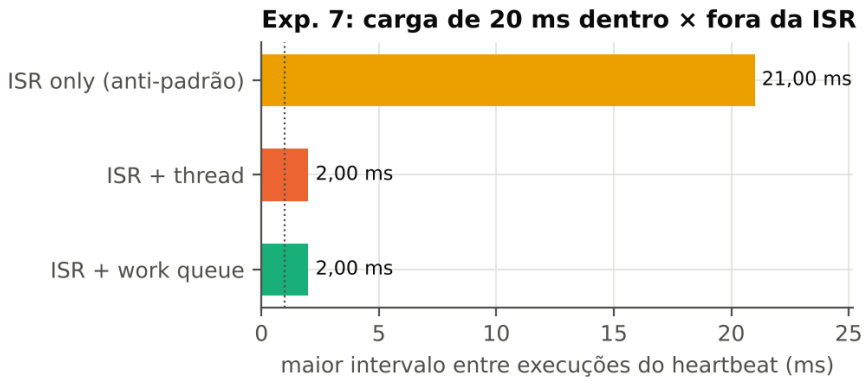


Figura 4 - Maior intervalo entre acordadas do heartbeat (prioridade 0, período nominal 1 ms). A linha pontilhada marca 1 ms.

Com os 20 ms de espera ocupada dentro da ISR, o heartbeat ficou até 21,00 ms sem rodar: enquanto a ISR executa, nem o tick nem o escalonador avançam. Com o mesmo trabalho numa thread ou na work queue, ambos de prioridade 1, o heartbeat (prioridade 0) os preempta e o maior intervalo fica em 2,00 ms e 2,00 ms, perto do baseline de 1 a 2 ticks.

Discussão obrigatória.

- **CPU do kernel no modo isr_only.** O kernel mediu 0,12 % contra 16,72 % estimados a partir das durações. O tempo gasto na ISR é contabilizado na thread interrompida, quase sempre a idle, e por isso aparece como "ocioso". A CPU estimada mede o tempo dentro de `app_process_event()` e não tem esse ponto cego.
- **Prioridades.** Uma thread de prioridade **maior** que a worker (como o heartbeat) não sofre com o processamento longo numa thread: ela a preempta. Uma de prioridade **menor** espera o tratamento terminar. Já uma ISR longa atrasa **todas** as threads, qualquer que seja a prioridade.
- **Outras interrupções.** Interrupções de prioridade igual ou menor (UART, timer do sistema, Wi-Fi) ficam pendentes durante a ISR longa. Isso pode estourar FIFOs, perder caracteres e pacotes e, no caso do timer, atrasar o tempo do kernel, como o heartbeat mostrou.

7.7 Experimento 8: desafio final (processamento de 100 ms)

As três versões pedidas estão em `codigo/01_polling`, `02_interrupt_thread` e `03_workqueue`. Cada uma detecta o botão, incrementa um contador, alterna o LED, imprime o contador fora da ISR (na thread poll, worker ou sysworkq), simula 100 ms de processamento e volta a esperar. Elas reaproveitam a instrumentação do projeto-base. O processamento usa `k_msleep(100)` (thread dorme) ou `k_busy_wait()` (consome CPU), escolhido por `CONFIG_APP_PROC_BUSY`. O estímulo (50 eventos) roda com prioridade 0, papel de um evento externo que o processamento não pode atrasar.

Tabela 14.1 - Desafio final: tempo, CPU e eventos

Versão	Cenário	Proc. / gerados	Perdidos (%)	Lat. process média (µs)	Lat. process máx (µs)	Tratamento (ms)	CPU ocupada	CPU estimada	Pilha evento (B)
Polling	sleep, 5 ev/s	50 / 50	0,0	4.700,46	10.000,01	101,0	0,01 %	49,74 %	576 / 1024
Polling	busy, 5 ev/s	50 / 50	0,0	7.999,68	8.000,01	100,0	49,27 %	49,25 %	576 / 1024
Polling	busy, 20 ev/s	17 / 50	66,0	4.940,53	10.000,00	100,0	64,14 %	64,13 %	576 / 1024
ISR + thread	sleep, 5 ev/s	50 / 50	0,0	6,71	6,72	101,0	0,00 %	49,74 %	560 / 1024
ISR + thread	busy, 5 ev/s	50 / 50	0,0	6,71	6,72	100,0	49,26 %	49,25 %	560 / 1024
ISR + thread	busy, 20 ev/s	26 / 50	48,0	26.988,67	99.189,95	100,0	98,08 %	98,08 %	560 / 1024
ISR + work queue	sleep, 5 ev/s	50 / 50	0,0	7,32	7,33	101,0	0,00 %	49,74 %	608 / 1024
ISR + work queue	busy, 5 ev/s	50 / 50	0,0	7,32	7,33	100,0	49,26 %	49,25 %	608 / 1024
ISR + work queue	busy, 20 ev/s	26 / 50	48,0	26.994,49	99.201,07	100,0	98,08 %	98,08 %	608 / 1024

sleep = `k_msleep(100)`; busy = `k_busy_wait(100000)`. 5 ev/s = período de 197 ms; 20 ev/s = período de 47 ms (49 ticks reais). 50 eventos por execução.

Tabela 14.2 - Desafio final: memória

Versão	Flash (B)	RAM estática total (B)	Thread/pilha do tratamento
Polling	146.432	20.680	poll: 1 KiB + k_thread
ISR + thread	146.576	20.712	worker: 1 KiB + k_thread + k_sem
ISR + work queue	146.512	20.752	sysworkq: 1 KiB + k_work_q (linkada por uso)

O que os dados mostram. (i) Com `k_msleep(100)` a CPU ocupada ficou em 0,00 % a 0,01 %, enquanto a CPU "estimada" a partir das durações ficou em 49,74 %: dormir não consome CPU. Com `k_busy_wait()` as duas medidas coincidem (49,26 %). (ii) A 5 ev/s (utilização de 50 %) nenhuma versão perdeu eventos. (iii) Na rajada (período de 47 ms, 49 ticks reais, ≈ 20 ev/s) a demanda é de ≈ 200 % de CPU. As versões por interrupção processaram 26 de 50 eventos com 98,08 % de CPU: a ISR detecta todos, e a coalescência descarta o excedente. O polling processou só 17: ele perde na detecção, porque não consulta o pino durante os 100 ms. (iv) O hand-off do modo B subiu de 3,32 µs (Exp. 4) para 5,60 µs, porque aqui a `main` tem prioridade 0 e termina `pulse()` antes de a worker rodar. Pelo mesmo motivo, no polling a `main` passa a rodar antes da `poll` no mesmo tick, e a latência mínima cai para 4,58 µs.

Escolha para produção: interrupção + thread dedicada (com `k_msgq` se cada evento precisar ser contado). Justificativa:

- **Tempo de resposta:** a detecção é de µs, contra P/2 no polling.
- **Processamento longo:** 100 ms bloqueando a sysworkq atrasariam todos os outros work items do sistema, o que a apostila desaconselha. A thread própria isola esse custo e permite escolher a prioridade.
- **Sequências de eventos:** com semáforo ou fila, a thread absorve rajadas curtas; o polling as perde na detecção.
- **Custo:** neste firmware a RAM estática da versão com thread (20.712 B) é praticamente a mesma da versão com work queue (20.752 B), porque a sysworkq também traz uma pilha de 1 KiB. Mesmo num sistema em que a sysworkq já exista, o custo extra seria de cerca de 1,2 KiB, aceitável num ESP32 com 320 KiB de DRAM.
- **Organização:** a ISR só sinaliza, e o tratamento fica em uma função de thread fácil de testar.

Uma work queue **dedicada** (E4) seria equivalente em custo e comportamento. A work queue do sistema só seria adequada se o tratamento fosse curto.

Tabela 14.3 - Decisão por cenário (seção 14.2 da apostila)

Cenário	Arquitetura escolhida	Justificativa
Botão que liga/desliga um LED (baixa taxa, tratamento trivial)	ISR + work queue (sistema)	Detecção imediata; tratamento de µs não prejudica a sysworkq; se ela já existe no sistema, custa só o k_work (16 B).
Sensor com 1.000 amostras/s e 50 µs por amostra	ISR + fila de mensagens + thread	5 % de CPU, mas cada amostra importa: a fila (k_msgq) evita a coalescência e absorve atrasos da thread.
Pacote de rede: filtrar, copiar e notificar (~2 ms)	ISR + thread (ou work queue dedicada) com fila	2 ms é longo para a sysworkq; a thread própria isola o custo e a fila preserva pacotes em rajada.
Verificar a cada 500 ms se a bateria caiu abaixo de um limite	Polling	Grandeza lenta e periódica, sem interrupção natural; polling de 500 ms custa ≈ 0 % de CPU.
Encoder de motor a 20 kHz (contar todos os pulsos)	Contador de hardware (PCNT) ou ISR mínima	50 µs entre pulsos inviabiliza thread por pulso (coalesceria); o PCNT do ESP32 conta sem CPU, ou a ISR só incrementa um contador atômico.
Botão de emergência que deve parar um atuador em < 1 ms	ISR que age diretamente	A ISR desliga o atuador (escrita em GPIO, poucos µs) e só depois sinaliza uma thread; polling exigiria P < 1 ms.

8. Análise comparativa e discussão crítica

Tabela 15.2 - Análise comparativa

Critério	Polling	ISR + thread	Work queue
Como detecta o evento?	Thread consulta o pino a cada P (efetivo P + 1 tick)	Interrupção de GPIO (borda)	Interrupção de GPIO (borda)
Como processa o evento?	Na própria thread de polling	Na thread worker, acordada por k_sem	No handler do k_work, na sysworkq
Existe ISR?	Não	Sim (curta)	Sim (curta)
Existe thread dedicada?	Sim (poll)	Sim (worker)	Não
Usa semáforo? Work item?	Nenhum	Semáforo (k_sem, limite 1)	Work item (k_work)
Precisa verificar continuamente?	Sim	Não	Não
Complexidade	Baixa	Média	Média
Latência média / máxima medida (até o tratamento)	9,00 / 9,00 ms (fase travada); 6,03 / 10,99 ms (fase varrida)	4,42 / 4,43 µs	5,03 / 5,04 µs
CPU ocupada medida (sem carga)	0,02 % (1.037.952 ciclos na poll)	0,01 % (151.616 ciclos na worker)	0,01 % (165.537 ciclos na sysworkq)
Memória (Flash / RAM estática total / pilha do evento, usada/total)	146.416 / 20.680 B / 256 / 1024	146.560 / 20.712 B / 288 / 1024	146.496 / 20.752 B / sysworkq 304 / 1024
Comportamento quando não há evento	Acorda a cada P + 1 tick	Bloqueada em k_sem_take()	Fila vazia; sysworkq bloqueada
Aplicação indicada	Grandezas lentas/periódicas, verificações simples	Tratamento longo ou contínuo dedicado ao evento	Trabalho curto e diferido

Discussão crítica. Os números confirmam a teoria em sinal e ordem de grandeza, e três pontos merecem destaque. Primeiro, o polling depende da relação entre os períodos *em ticks*, não só de P. O arredondamento de +1 tick do `k_msleep()` tornou o período de 97 ms um múltiplo exato do período efetivo de polling, e a latência ficou travada em 9 ms em vez de se distribuir entre 1 e P + 1 ticks. Já com P próximo do período do estímulo, o aliasing fundiu eventos e elevou a perda de 79 % (previsto) para 95,5 %. Escolher um período "primo em ms" não basta: o que importa é que ele não tenha divisor comum com o laço de polling medido em ticks efetivos, e que o laço seja bem mais curto que o intervalo entre eventos. Segundo, a vantagem de memória da work queue pressupõe que a sysworkq já exista; no Zephyr 4.5 ela é linkada sob demanda, e num app isolado custa o mesmo que uma thread dedicada. Terceiro, as diferenças de CPU sem carga são de centésimos de %. O que distingue as arquiteturas é como o custo **escala**: com 1/(P + 1 tick) no polling, com a taxa de eventos nas outras duas. Os ciclos acumulados por thread mostram isso melhor que a porcentagem arredondada do Thread Analyzer. A medida externa pelos pinos concordou com a medida interna (Tabela 3.1).

9. Respostas das questões (capítulo 18)

1. **Principal diferença entre polling e interrupção?** No polling o software pergunta periodicamente; na interrupção o hardware avisa. Medido: 9,00 ms (P = 10) contra 1,10 µs de latência de detecção.
2. **Por que o polling consome processamento sem eventos? Apareceu nas medições?** A thread acorda a cada P + 1 tick, troca de contexto e lê o pino mesmo sem evento. Sim: a `poll`

acumulou 1.037.952 ciclos contra 151.616 da `worker`, e no Exp. 5 a CPU ocupada foi de 0,12 % (P = 1) a 0,00 % (P = 100).

3. **Função da ISR e três coisas que ela não deve fazer?** Reconhecer o hardware e sinalizar o evento rapidamente (registrar tempo, contar, dar o semáforo ou submeter o work). Não deve bloquear ou dormir (k_msleep, k_sem_take com espera, k_mutex_lock), executar laços ou cálculos longos, nem chamar APIs não seguras para ISR ou printk.
4. **Por que a ISR não deve fazer processamento demorado? Evidência do Exp. 7?** Porque bloqueia o escalonador, o tick e as interrupções de prioridade igual ou menor. No Exp. 7 o heartbeat ficou até 21,00 ms sem rodar com 20 ms na ISR, contra 2,00 ms com o mesmo trabalho numa thread.
5. **Função do semáforo na implementação com thread?** Comunicar à thread, sem espera ativa, que o evento ocorreu. A worker fica bloqueada em k_sem_take() e acorda com k_sem_give() da ISR. O limite 1 funde sinais repetidos.
6. **Função da work queue e o que é a system workqueue?** Executar trabalho diferido em contexto de thread. A system workqueue é uma thread do kernel (sysworkq), compartilhada com drivers e subsistemas, que executa em ordem os work items submetidos.
7. **Diferença entre k_sem_give() e k_work_submit()?** k_sem_give() libera um semáforo que uma thread sua aguarda; k_work_submit() enfileira um item cujo handler roda na work queue. Ambos são seguros em ISR. Hand-off medido: 3,32 µs contra 3,93 µs.
8. **Em qual implementação há thread dedicada? Custo em memória?** No modo B (worker) e também no polling (poll). A thread dedicada custa a pilha de 1024 B + k_thread de 184 B + k_sem de 16 B (a pilha usou 288 / 1024 B). Medido: a RAM estática total do modo B (20.712 B) ficou 40 B abaixo da do modo C, porque no Zephyr 4.5 a sysworkq só é linkada quando usada e traz sua própria pilha de 1 KiB.
9. **O que acontece com eventos gerados rapidamente? Como os dados mostram (Exp. 6B)?** São coalescidos: com 200 bordas detectadas, a thread processou 167 e a work queue 167; o excedente virou perda no encaminhamento.
10. **O polling pode deixar de perceber um evento? Cenário do Exp. 5?** Sim: quando o pulso é mais curto que o período efetivo ou cai entre duas consultas. Com pulso de 5 ms, P = 10 perdeu 46,0 % e P = 100 perdeu 96,0 %. No modo manual, toques de 4 ms também foram perdidos.
11. **Como a prioridade afeta o hand-off? Por que igualar worker e sysworkq?** O hand-off só ocorre quando a thread é a de maior prioridade pronta. Com a main na prioridade 0 (Exp. 8) o hand-off do modo B subiu de 3,32 para 5,60 µs. Igualar as prioridades garante que B e C sejam comparados só pelo mecanismo.
12. **A CPU ocupada do kernel é confiável para ISRs longas?** Não. O tempo da ISR é atribuído à thread interrompida (normalmente a idle). No Exp. 7 o kernel marcou 0,12 % contra 16,72 % estimados pelas durações.
13. **Por que a main gera o estímulo em prioridade baixa?** Para que a worker (prioridade 1) a preempte imediatamente após a ISR; assim o hand-off não inclui o restante de pulse(). O efeito medido foi de 2,29 µs (questão 11). O preço é que, com carga, o período do estímulo cresce (Exp. 6A).
14. **Fontes de erro nas medições de tempo?** Viés de gpio_pin_set_dt() antes da borda (Tabela 3.1), resolução de 4,17 ns e estouro do contador de 32 bits, printk na janela, cache e primeiras execuções (mitigados pelo aquecimento), tick de 1 ms e arredondamento +1 tick, coalescência e a atribuição do tempo de ISR.
15. **Processamento de 100 ms e 5 eventos/s? E 50 eventos/s?** A 5 ev/s a utilização é de 50 %: thread dedicada (ou work queue dedicada) com fila. No Exp. 8 não houve perdas, com CPU de

49,26 % em espera ocupada. A 50 ev/s (500 %) o sistema satura: nenhuma arquitetura resolve. Já a 20 ev/s o Exp. 8 perdeu 48,0 % na versão com thread. É preciso reduzir o processamento, usar mais CPU (o segundo núcleo) ou descartar e amostrar.

Questões dos Experimentos 1 a 3

Exp. 1. (1) O programa verifica repetidamente porque não há outro aviso de que o pino mudou. (2) Com P = 100 ms a latência pode chegar a 101 ms (média esperada ≈ 51 ms) e a maioria dos pulsos curtos passa despercebida: no Exp. 5, P = 100 perdeu 95,5 % dos pulsos de 20 ms. (3) A vantagem é a simplicidade e a previsibilidade; a desvantagem é a latência de até P e o custo fixo de CPU. (4) Sim: a thread executa a cada P mesmo sem toque, o que aparece nos ciclos acumulados da `poll`, embora o Thread Analyzer arredonde para 0 %. (5) `k_msleep()` bloqueia a thread e libera a CPU para a idle; um `while(1)` sem espera ocuparia 100 % da CPU.

Exp. 2. (1) A ISR detecta; a worker processa. (2) O semáforo sinaliza à worker sem espera ativa. (3) Para não bloquear o sistema (Exp. 7). (4) Não: a thread fica bloqueada em `k_sem_take()` e não consome CPU. (5) Com limite 1, dois eventos antes de a thread rodar viram um só processamento (Exp. 6B).

Exp. 3. (1) A work queue executa trabalho diferido em contexto de thread. (2) Ver questão 7 do capítulo 18. (3) A thread dedicada está no modo B; no C o tratamento roda na sysworkq. (4) O `k_work_submit()` de um item pendente retorna 0 e o evento é coalescido. (5) Porque a sysworkq é compartilhada: bloqueá-la atrasa todos os outros work items do sistema.

10. Conclusão e recomendação

As três arquiteturas resolvem o problema, mas com custos de naturezas diferentes. O polling é o mais simples e o único sem ISR, mas paga em latência (≈ P/2 + 1 tick, chegando a 9 ms fixos por trava de fase com os parâmetros padrão), em CPU proporcional a 1/(P + 1 tick) e em eventos perdidos quando o pulso é mais curto que o período ou o período se aproxima do intervalo entre eventos. As arquiteturas por interrupção detectam em 1,10 µs, não gastam CPU sem eventos e só perdem eventos por coalescência, quando a taxa supera a capacidade de tratamento. Entre elas, a thread com semáforo tem hand-off ligeiramente menor e isola o processamento. A work queue do sistema só é mais econômica em RAM quando a sysworkq já existe por outros motivos (aqui ela custou 40 B a mais que a thread) e só serve para trabalho curto. O anti-padrão de processar na ISR foi demonstrado com dados: 21,00 ms de bloqueio do escalonador e uma CPU do kernel que esconde a carga.

Recomendação por cenário: polling para grandezas lentas e periódicas; ISR + work queue do sistema para tratamentos curtos e esporádicos; ISR + thread (ou work queue dedicada) para tratamentos longos como o de 100 ms; ISR + `k_msgq` quando cada evento importa; e periféricos dedicados (PCNT) ou ISRs mínimas para eventos de alta frequência.

Apêndice A: arquivos entregues

Tabela A.1 - Estrutura da entrega

Caminho	Conteúdo
codigo/00_bench_eventos/	Projeto-base do curso (Exps. 1 a 7), sem alterações
codigo/01_polling/, 02_interrupt_thread/, 03_workqueue/	Três versões do desafio final (Exp. 8)
codigo/exp0_blink_test/	Validação do ambiente (Exp. 0)
logs/*_runN.log	Logs brutos de cada execução (saída serial completa, com as linhas CSV)
logs/*_build.txt, *_ram.txt, *_rom.txt, *_secoes.txt	Saída do west build (tabela de memória), relatórios de RAM/ROM e seções do ELF
logs/*_gpio_trace.csv	Registro das transições dos pinos 2, 4 e 18 (Exps. 1 a 4)
resultados.xlsx	Dados extraídos dos logs, tabelas da apostila (fórmulas) e gráficos
emulacao/	Ferramentas de execução e análise (Apêndice B)

Apêndice B: detalhes da execução

Plataforma. O firmware compilado para a `esp32_devkitc` foi executado no QEMU 9.2.2 da Espressif (`esp-develop`, commit `febae18`). O modelo de GPIO desse QEMU só responde ao registrador de *strapping*; ele foi estendido (`emulacao/qemu-esp32-gpio.patch`) com os registradores `OUT`, `ENABLE`, `IN`, `STATUS`, `PCPU_INT` e `PINn` usados pelo driver `gpio_esp32.c`, a detecção de borda/nível ligada à fonte 22 (`ETS_GPIO_INTR_SOURCE`) da matriz de interrupções, o jumper GPIO18 → GPIO4, a sequência de toques do botão e o registro das transições dos pinos. O eFuse padrão do emulador informa revisão de chip v0.0, aceita por `CONFIG_ESP32_USE_UNSUPPORTED_REVISION=y`, já presente no `prj.conf` do curso.

Tempo. Com `-icount shift=2,align=off,sleep=off` cada instrução avança 4 ns do relógio virtual, e o CCOUNT é derivado desse relógio a 240 MHz (≈ 0,96 ciclo por instrução); com a CPU ociosa o relógio avança direto até o próximo timer. A execução é determinística: as 3 execuções de cada configuração produziram os mesmos números (amplitude da média de `lat_process` entre execuções: 0,00 / 0,00 / 0,00 µs).

Sensibilidade ao CPI. Como o emulador não modela cache nem estados de espera da flash, o Exp. 4 foi repetido com 8 ns por instrução (≈ 2 ciclos por instrução, `shift=3`):

Tabela B.1 - Exp. 4 com CPI ≈ 2 (1 execução)

Métrica (µs)	Polling	ISR + thread	ISR + work queue
lat_detect: média	8.999,84	2,20	2,20
lat_handoff: média	n/a	6,64	7,86
lat_process: média	8.999,84	8,85	10,07

A latência do polling, dominada pelos ticks, não muda; as latências por interrupção dobram. Os valores de µs da seção 7 correspondem, portanto, ao limite inferior dessa faixa; em placa haveria ainda variação de execução para execução por efeitos de cache. Os resultados que dependem de ticks, prioridades, escalonamento e coalescência não são afetados.

Reprodução. Aplicar `qemu-esp32-gpio.patch` ao QEMU da Espressif, compilar `xtensa-softmmu` e executar `python3 emulacao/run_lab.py exp4` (ou outro prefixo) num workspace west com o manifesto do curso; `analisar.py`, `figuras.py` e `gerar_relatorio.py` regeneram a planilha, as figuras e este relatório. Em uma placa, os mesmos projetos são compilados e gravados com `west build` e `west flash`, como na apostila.